IABot Chat Application

Powered by **gemini-2.0-flash-001**

Inamullah [023-23-0048] & Ali Tawassul [023-23-0061]

Batch: BS - IV

Course: Database System

Instructor: Dr. Adil Khan

Submission Date: 10th June 2025

Project Overview

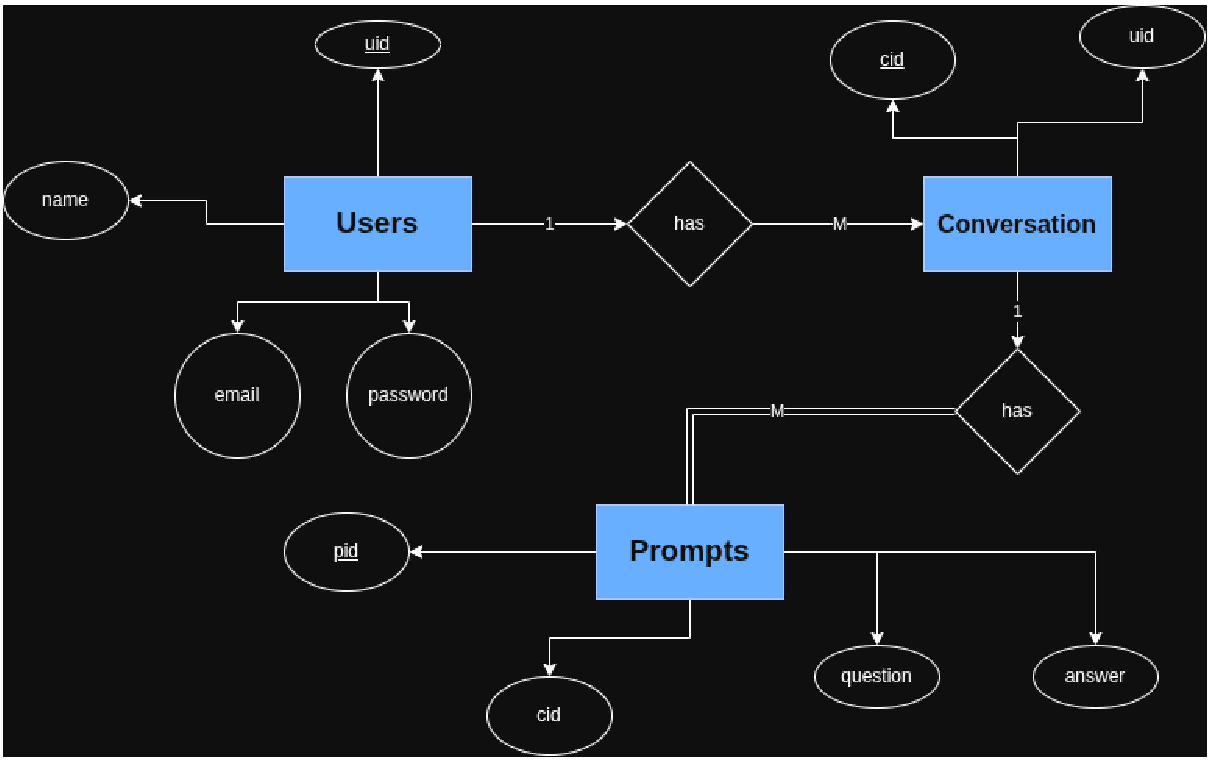
The **IABot Chat Application** is a desktop-based interactive chatbot system that leverages a structured and normalized relational database to manage user interactions. This project is primarily focused on demonstrating robust database design, implementation, and integration principles. The front-end is built using **Java Swing**, while the backend is powered by **SQLite**.

A major feature of this chatbot is its integration with the **Gemini 2.0 API**, a powerful generative AI model. User prompts are sent to the Gemini model, which returns intelligent, contextual responses. These interactions are persistently stored in the database, allowing users to retain full conversation histories.

Key goals achieved in this project include:

- Designing an entity-relationship model and transforming it into a normalized schema.
- Implementing multi-user authentication and conversation management.
- Persistently saving each prompt and AI-generated response using relational principles.
- Ensuring data integrity through the use of primary/foreign keys and constraints.
- Providing a responsive, user-friendly interface for CRUD operations.

1. Entity Relationship Diagram (ERD)



2. Normalization

Unnormalized Form (UNF)

Original data with redundancy:

Chat(uid, name, email, password, cid, title, pid, question, answer)

This structure contains repeated and non-atomic values and violates the basic principles of normalization.

1NF - First Normal Form

- Eliminate repeating groups and non-atomic attributes.
- Separate data into individual tables.

Tables:

Users(uid, name, email, password)
Conversations(cid, uid, title)
Prompts(pid, cid, question, answer)

Each attribute is now atomic, and rows are uniquely identifiable.

2NF - Second Normal Form

- Eliminate partial dependencies.
- Ensure every non-key attribute is fully functionally dependent on the whole primary key.

Improvements:

- Prompts are now linked only by `cid`, ensuring full dependency.
- Conversations are linked directly to users.
- Users table is self-contained.

3NF - Third Normal Form

- Eliminate transitive dependencies.
- Ensure all non-key attributes are dependent only on the primary key.

No transitive dependencies were found in the normalized structure. All three tables meet the 3NF criteria, resulting in improved data consistency and reduced redundancy.

3. DBMS and Programming Language Integration

Database Used: SQLite (sqlite3)

- Lightweight and file-based.
- Ideal for desktop applications.
- No server setup required.

Programming Language: Java with Swing

- Java is used to build the GUI and backend logic.
- JDBC driver used for SQLite database interaction.

Libraries Used:

- sqlite-jdbc-3.43.2.0.jar (JDBC Libraries to connect Java with Database)
- slf4j-api-2.0.9.jar
- slf4j-simple-2.0.9.jar

Database-Oriented Application Features:

- **Authentication System:** Uses the Users table to validate login and register new users.
- **Conversation Management:** Each conversation is linked to a user (FK), allowing multi-user support.
- **Prompt Storage:** Each user message and AI response is saved in the Prompts table with an FK to the Conversations table.

CRUD Operations:

The IABot application provides a fully integrated experience where the relational database system directly supports and interacts with the graphical user interface (GUI). The GUI enables seamless access to all CRUD functionalities:

- **Create:** Users can register accounts, create conversations, and submit prompts.
- **Read:** Conversations and responses are dynamically loaded based on selection.
- **Update:** Titles of conversations can be renamed directly via the GUI.
- **Delete:** Conversations can be removed, triggering a cascade delete of their prompts.

Snapshots and Video of the GUI interface (included below) visually demonstrate how each database operation is reflected in real-time. This bridge between frontend usability and backend integrity underlines the practical utility of relational database systems in interactive applications.

Database Validation & Error Handling:

- Prevents empty inserts.
- Ensures referential integrity via foreign key constraints.
- Handles exceptions with clean UI messages.

4. Final View of the Database

Tables & Fields:

```
sqlite> .tables
Conversations  Prompts        Users
```

```
sqlite> .schema Conversations
CREATE TABLE Conversations (
    cid INTEGER PRIMARY KEY AUTOINCREMENT,
    uid INTEGER NOT NULL, title TEXT,
    FOREIGN KEY (uid) REFERENCES Users(uid) ON DELETE CASCADE
);
sqlite>
```

```
sqlite> .schema Users
CREATE TABLE Users (
    uid INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL,
    password TEXT NOT NULL
);
```

```
sqlite> .schema Prompts
CREATE TABLE Prompts (
    pid INTEGER PRIMARY KEY AUTOINCREMENT,
    cid INTEGER NOT NULL,
    question TEXT NOT NULL,
    answer TEXT NOT NULL,
    FOREIGN KEY (cid) REFERENCES Conversation(cid) ON DELETE CASCADE
);
sqlite>
```

Sample Records:

```
sqlite> SELECT * FROM Users;
101|inam|inam@admin.com|123
102|ali|ali@admin.com|456
103|fahad|fahad@gmail.com|fahad
sqlite>
```

```
sqlite> SELECT * FROM Conversations;
2|101|first chat
4|101|personal talks
6|101|ia
7|102|hello world
8|102|cim
9|103|quickchat
10|103|new
sqlite> SELECT * FROM Promptss;
```

```
sqlite> SELECT * FROM Prompts;
1|2|hello|Okay, I will. What do you need?

2|2|Explain Love|Love is a strong affection and care for another, often involving deep emotional connection, attraction, and a
 desire for their well-being and happiness.

3|2|Who is the founder of Pakistan?|Muhammad Ali Jinnah.

4|2|Which material is used while creating teeth?|Dental porcelain or composite resin.

5|2|Thank You|You're welcome.

6|2|OK|Understood.
```

Constraints:

- **Primary Keys:** uid, cid, pid
- **Foreign Keys:**
    - Conversations.uid → Users.uid
    - Prompts.cid → Conversations.cid

Cascading Rules:

- ON DELETE CASCADE from Conversations to Prompts.

- Ensures that deleting a conversation removes its related prompts.

SQL Sample Queries:

-- Get all user conversations

SELECT * FROM Conversations WHERE uid = 1;

-- Insert new prompt

INSERT INTO Prompts (cid, question, answer) VALUES (?, ?, ?);

-- Rename a conversation

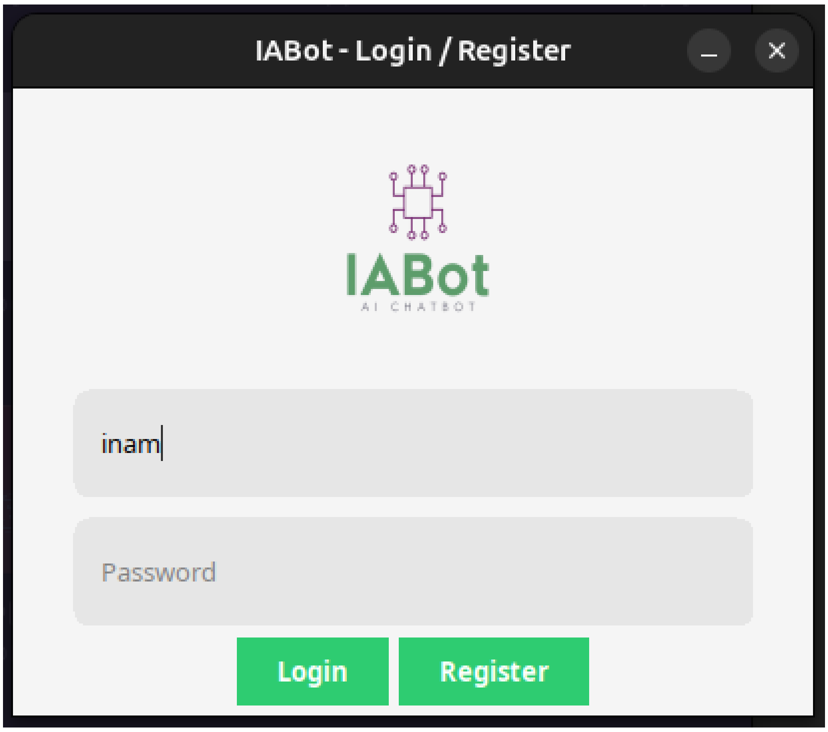
UPDATE Conversation SET title = ? WHERE cid = ?;

-- Authenticate user

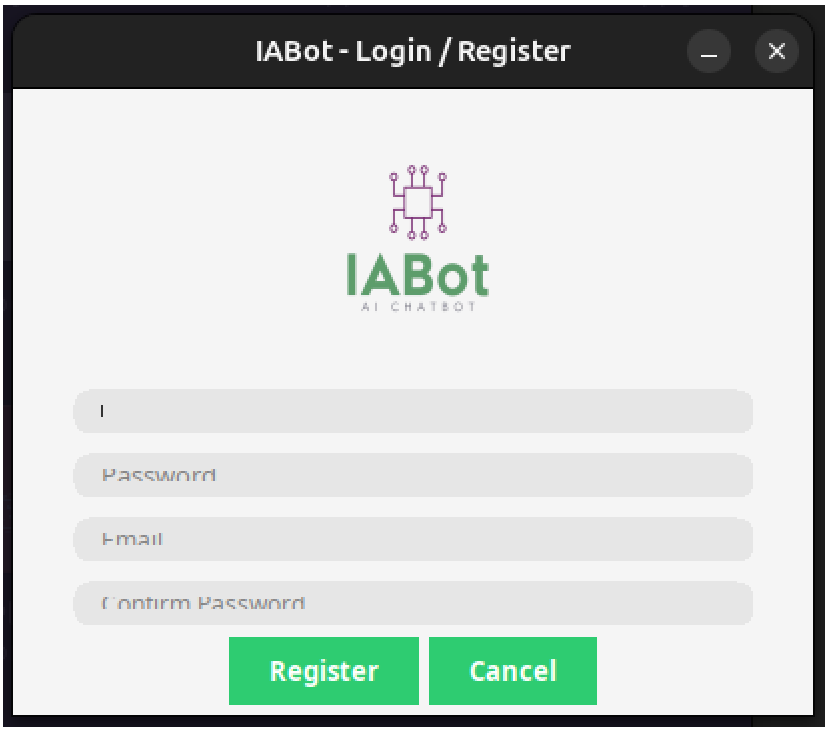
SELECT uid FROM Users WHERE name = ? AND password = ?;

Application Snapshots

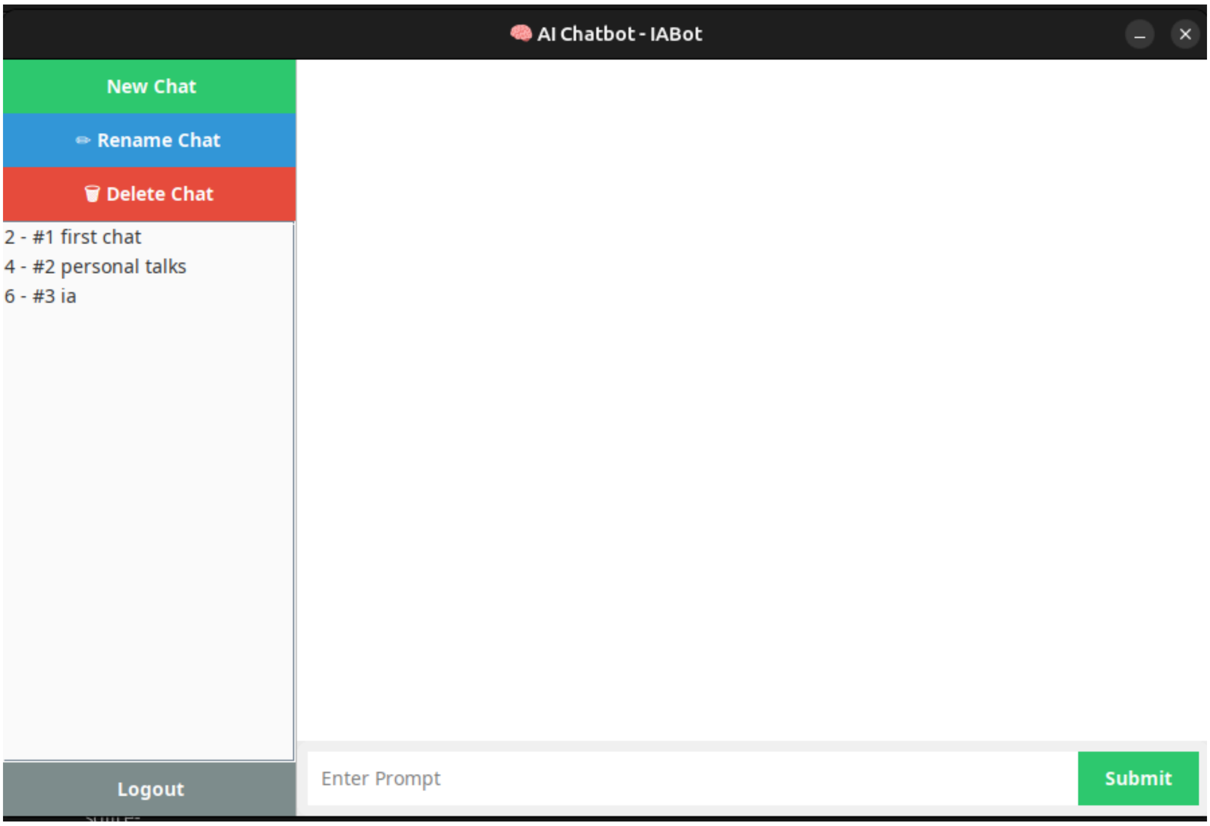
Login Page:

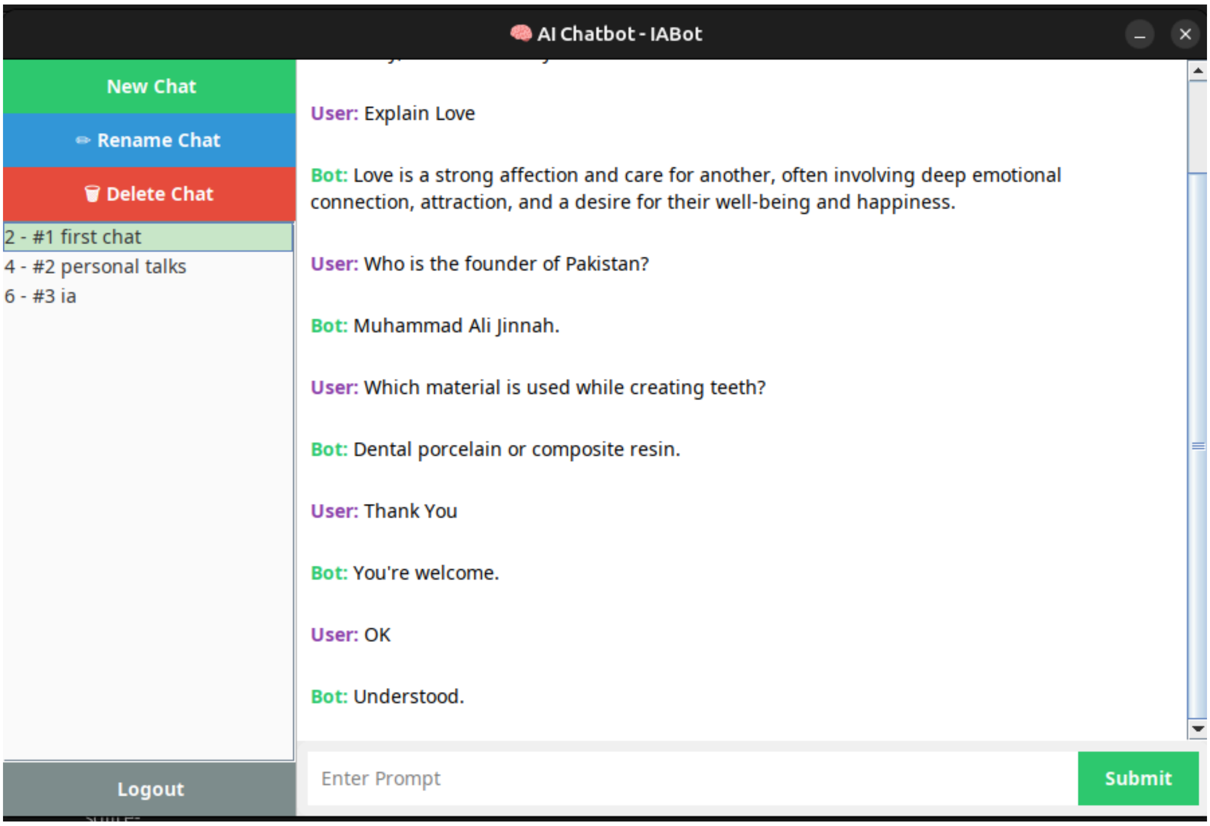


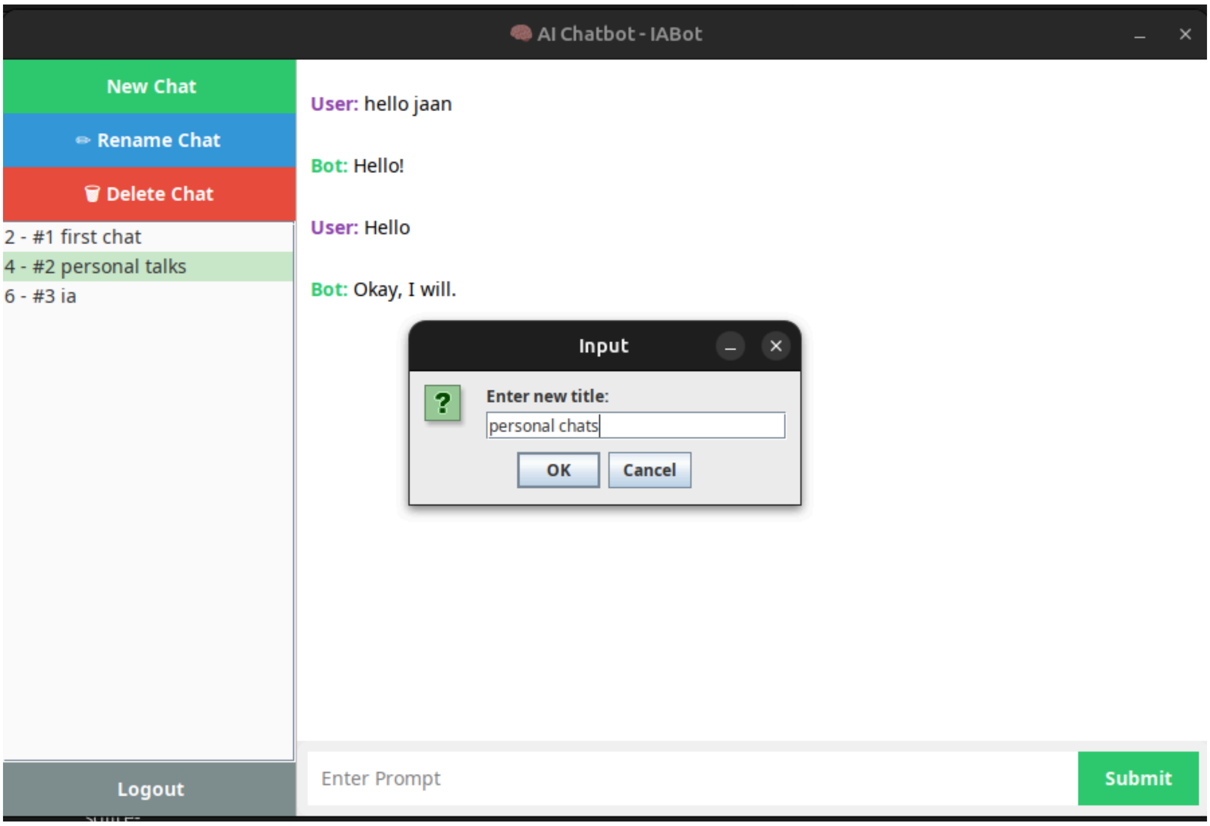
Register Page:



ChatApp Page: (Main)




AI Chatbot - IABot
New Chat
✏ Rename Chat
Delete Chat
2 - #1 first chat
4 - #2 personal talks
6 - #3 ia
Logout
User: Explain Love
Bot: Love is a strong affection and care for another, often involving deep emotional connection, attraction, and a desire for their well-being and happiness.
User: Who is the founder of Pakistan?
Bot: Muhammad Ali Jinnah.
User: Which material is used while creating teeth?
Bot: Dental porcelain or composite resin.
User: Thank You
Bot: You're welcome.
User: OK
Bot: Understood.
Enter Prompt
Submit


AI Chatbot - IABot
New Chat
✏ Rename Chat
Delete Chat
2 - #1 first chat
4 - #2 personal talks
6 - #3 ia
Logout
User: hello jaan
Bot: Hello!
User: Hello
Bot: Okay, I will.
Input
Enter new title:
personal chats
OK
Cancel
Enter Prompt
Submit

Conclusion

This project successfully demonstrates the integration of a relational database system with a functional GUI application. The database design adheres to normalization standards (3NF) and incorporates essential constraints and relationships. With full CRUD support, error handling, and interface integration, the system meets the practical requirements of a real-world multi-user chatbot.